Int. j. inf. tecnol. (March 2025) 17(2):925–939
https://doi.org/10.1007/s41870-024-02260-8

ORIGINAL ARTICLE

MapReduce model for efficient image retrieval: a Hadoop-based framework

Maher Alrahhal[1] · **Vinod Kumar Shukla**[1,2]

Received: 26 April 2024 / Accepted: 23 October 2024 / Published online: 7 December 2024
© Bharati Vidyapeeth's Institute of Computer Applications and Management 2024

Abstract The rapid proliferation of images, driven by advancements in image-capturing technologies, poses significant challenges to the efficient management and retrieval of images from vast databases. Traditional Content-Based Image Retrieval (CBIR) systems struggle with scalability and complexity, resulting in suboptimal retrieval performance. To address these challenges, this paper explores the integration of CBIR with Hadoop, a well-established distributed computing framework. Hadoop's capability to handle large-scale data processing, utilizing its MapReduce model and Hadoop Distributed File System (HDFS), offers a promising solution to enhance the effectiveness and scalability of image retrieval tasks. Our solution combines the strengths of CBIR and Hadoop, using our proposed Full Directional Local Neighbor Pattern (FDLNP) method for feature extraction. This method captures local patterns, color, texture, and directional information to provide a comprehensive image representation, significantly improving retrieval accuracy. We present a detailed design and implementation of this integrated system, emphasizing its two main phases: the offline phase, which constructs the feature database using FDLNP and MapReduce jobs, and the online phase, which extracts features from query images and calculates similarity distances using parallel processing. Experimental results on a Hadoop cluster reveal a significant improvement in processing efficiency, particularly for large datasets, highlighting the advantages of distributed processing in managing extensive image retrieval tasks. The findings indicate that while single-node systems may be suitable for smaller datasets, Hadoop clusters are preferable for larger-scale image databases due to their scalability and concurrent processing capabilities. Integrating CBIR with Hadoop, enhanced by our FDLNP method, provides a powerful tool for organizing and searching images in vast databases, offering improved retrieval performance, scalability, fault tolerance, and reliability—key attributes for large-scale image retrieval systems.

Keywords CBIR · Hadoop distributed computing framework · Big data processing · MapReduce model · Hadoop distributed file system (HDFS) · CBIR with Hadoop integration

1 Introduction

In the digital age, the unprecedented proliferation of imagery—driven by rapid advancements in technologies such as smartphones, digital cameras, and the omnipresence of social media platforms—has resulted in a vast surge of digital image data. This deluge of imagery challenges traditional paradigms of image storage, management, and retrieval, rendering conventional Content-Based Image Retrieval (CBIR) systems increasingly ineffective.

CBIR systems are designed to identify relevant images within large databases based on content characteristics such as colors, shapes, textures, and spatial arrangement. CBIR operates in two main phases: the offline phase, which involves preprocessing images, extracting features to accurately represent each image, and effectively storing these features; and the online phase, which focuses on processing user queries, matching the query image against the database

✉ Maher Alrahhal
malrahhal@amitydubai.ae

1 Postdoctoral Fellow, School of Engineering, Architecture, and Interior Design, Amity University Dubai, Dubai, United Arab Emirates

2 Associate Professor, School of Engineering, Architecture, and Interior Design, Amity University Dubai, Dubai, United Arab Emirates

using similarity measures, and presenting the most relevant images to the user. However, as image repositories grow in complexity and volume, CBIR systems face significant challenges in scalability and maintaining high retrieval accuracy. The pressing need for robust, scalable, and efficient image retrieval solutions is underscored by these evolving challenges [1–5].

To address these limitations, CBIR systems have increasingly turned to distributed computing frameworks. Hadoop has emerged as a prominent solution for processing large-scale datasets, particularly in the terabyte and petabyte ranges. It achieves this by distributing computational tasks across clusters of commodity hardware, utilizing the Hadoop Distributed File System (HDFS) and the MapReduce framework for efficient data storage and parallel processing [6]. By integrating CBIR with Hadoop, we aim to leverage its distributed processing capabilities to enhance scalability and improve retrieval rates in image retrieval tasks [7]. This integration facilitates the parallel processing of images from massive databases, resulting in significant improvements in retrieval performance.

A body of research has explored diverse approaches to enhance CBIR systems through distributed computing. For instance, Huang and Wu [8] introduced a Hadoop-based image retrieval system employing ASIFT features, showcasing enhanced image matching capabilities across varied conditions. Similarly, Dhulavvagol et al. [9] extended the utility of Hadoop to the medical domain, advocating for a framework conducive to large-scale medical data management, inclusive of efficient image storage and retrieval functionalities. Kavitha et al. [10] further examined the integration of CBIR with Hadoop using the Hadoop Image Processing Interface (HIPI), illustrating its potential for effective large-scale image processing. These studies illuminate the transformative potential of Hadoop in overcoming the scalability and performance limitations of traditional CBIR systems. Nonetheless, they also highlight gaps in computational efficiency and the integration of advanced feature extraction techniques within distributed environments.

Despite these advancements, several critical challenges remain. CBIR systems continue to grapple with computational inefficiencies when processing large datasets, the seamless integration of advanced feature extraction methods within distributed computing frameworks to bolster retrieval accuracy, and the full utilization of Hadoop's parallel processing capabilities to enhance scalability and retrieval performance.

In response, our research proposes a novel integration of CBIR with Hadoop, articulated through a meticulously designed two-phase approach. This method includes an offline phase to construct the feature database using Hadoop's MapReduce model, significantly improving the efficiency of image data processing. The online phase introduces an innovative retrieval process that leverages Hadoop's parallel processing capabilities to boost retrieval speed and accuracy. Additionally, this approach integrates cutting-edge feature extraction techniques within a distributed processing environment to ensure enhanced retrieval accuracy and relevance. The findings indicate that while a single-node system might suffice for smaller datasets, a Hadoop cluster is preferable for larger datasets due to its scalability and concurrent processing capabilities.

2 Related work

The domain of CBIR has evolved significantly over the past decades, particularly in handling large-scale image datasets. Initial methods primarily relied on low-level features such as color histograms, textures, and shapes for image representation and retrieval. However, with the explosion of image data generated by the rapid advancement of imaging technologies, these traditional methods have struggled with issues related to scalability, efficiency, and accuracy. In response, various research efforts have attempted to leverage distributed computing frameworks, like Hadoop, to address these limitations.

Huang and Wu [8] introduced a novel image retrieval system employing Hadoop to handle the computational complexity of large-scale image retrieval. By integrating Affine-SIFT (ASIFT) features and employing the Bag-of-Words (BoW) model with a Support Vector Machine (SVM) for image classification, they achieved improved image retrieval accuracy and robustness under various conditions. However, their approach suffers from high computational intensity, particularly with ASIFT feature extraction, and may not generalize well across diverse datasets due to its dependency on specific feature types.

Dhulavvagol et al. [9] presented a Hadoop-based framework for managing large-scale medical data, highlighting the potential of Hadoop's distributed architecture for efficient image storage and retrieval. Their approach achieved a 20% improvement in retrieval speed by distributing data storage and processing tasks across multiple nodes. While effective in a medical context, their work primarily focuses on data storage and lacks a comprehensive feature extraction and retrieval process tailored to CBIR.

Kavitha et al. [10] explored the use of the Hadoop Image Processing Interface (HIPI) for effective CBIR in large datasets. The authors utilized the MapReduce framework to process image data in parallel, achieving scalability and improved retrieval performance. Despite the success of their system, real-time processing capabilities and performance optimization remain open challenges due to the overhead associated with Hadoop's parallel data processing.

Another notable contribution is by Yousefnezhad et al. [11], who developed a CBIR framework using Hadoop's MapReduce for increased scalability and rapid response times. They introduced a unique bag-of-visual-words approach with a chain-clustering binary search-tree algorithm for efficient image retrieval. Their method excelled in recall, F-measure, and average precision, showcasing the potential of integrating advanced clustering techniques in CBIR. However, their approach introduces complexity in implementation, and its adaptability to non-image data processing remains unexplored.

Hinge et al. [12] tackled the problem of large-scale image retrieval by developing a Hadoop MapReduce-based framework that leverages parallel processing to handle extensive internet image datasets. Their architecture efficiently processes distributed image data but encounters challenges in managing varied image qualities and maintaining a user-friendly interface for accessibility. This method underscores the importance of scalable and effective tools for image retrieval in the modern "big data" era.

In the healthcare sector, Grace et al. [13] proposed a medical image retrieval system based on grid architecture and implemented using the Hadoop framework. This system enhanced the efficiency of storing and retrieving large quantities of medical images through CBIR algorithms. Their solution significantly reduced retrieval time, although issues related to urgent scenarios requiring faster processing speeds and compatibility with diverse image formats were highlighted.

Jai-Andaloussi et al. [14] introduced an efficient and scalable CBIR system for large-scale medical image databases using the Hadoop framework. They utilized new methods, BEMD-GGD and BEMD-HHT, to describe the content of medical images. Extensive testing on the Digital Database for Screening Mammography validated the practicality of their approach. While the integration of advanced feature extraction techniques with Hadoop's parallel processing capabilities proved effective, the specificity of their algorithms and resource-intensive processing make it less applicable to other types of image data.

Several studies have also explored the application of deep learning techniques, such as Convolutional Neural Networks (CNNs) and transfer learning, for feature extraction in CBIR systems. These methods have shown promising results in improving retrieval accuracy by learning high-level features directly from image data. However, their integration into distributed computing frameworks like Hadoop is still in its nascent stages, with challenges such as model deployment, parallel training, and real-time processing remaining largely unexplored.

Despite the strides made in employing Hadoop for image retrieval tasks, existing research identifies gaps in computational efficiency, scalability, and the seamless integration of advanced feature extraction methods within distributed environments. Most methods rely on single or limited feature extraction techniques, which may not fully capture the intricate characteristics of diverse image datasets, leading to suboptimal retrieval performance.

Our research addresses these challenges by introducing a novel integration of CBIR with the Hadoop framework, leveraging the Full Directional Local Neighbor Pattern (FDLNP) method for comprehensive feature extraction. By implementing a two-phase approach consisting of an optimized feature database construction process and an online retrieval phase, we harness Hadoop's distributed processing capabilities to enhance scalability, retrieval accuracy, and efficiency. Unlike previous works, our approach integrates cutting-edge feature extraction methods within a parallel processing environment, allowing for improved image retrieval performance, particularly for large-scale datasets. Additionally, our experiments compare the efficiency of using a single-node system against a Hadoop cluster, highlighting the conditions under which distributed processing becomes necessary and advantageous.

In summary, the integration of CBIR with Hadoop-based frameworks has shown promise in overcoming the limitations of traditional CBIR systems. However, further research is needed to explore the potential of combining advanced feature extraction techniques, such as FDLNP, with distributed computing to create a scalable, efficient, and highly accurate image retrieval system. Our study contributes to this field by proposing an innovative, Hadoop-integrated CBIR approach, offering enhanced retrieval speed, scalability, and fault tolerance suitable for managing large-scale image databases. An analytical comparison of the studies on image retrieval utilizing Hadoop is presented in Table 1.

3 Hadoop fundamentals

Hadoop is a Java-based open-source framework developed as part of an Apache project, designed to store and process large-scale data in a distributed environment. It uses the MapReduce framework to efficiently process vast volumes of data, enhancing performance through the principle of data locality—bringing processing units to where the data resides, which minimizes network overhead [15, 16].

Hadoop's key advantage lies in its linear scalability; as more nodes are added to the cluster, the platform efficiently manages both data and hardware resources, directly improving performance. The system comprises two main components: MapReduce, which handles the computational tasks, and the HDFS, which manages data storage and provides replication for fault tolerance [17].

Table 1 Comparative Overview of Hadoop-based Image Retrieval Studies

REF	Focus area	Key methods/techniques	Main findings	Limitations	Applications
[8]	Hadoop-based Image Retrieval	ASIFT, MapReduce, BoW, SVM	Enhanced retrieval efficiency and accuracy	Computational intensity of ASIFT; Generalization across diverse datasets	Digital libraries, medical imaging
[9]	Hadoop Framework for Medical Data	Hadoop, Distributed Architecture	20% improvement in storage and retrieval	Integration with existing systems; Data privacy and security concerns	Medical field, cloud-based systems
[10]	CBIR in Big Data with Hadoop and HIPI	Hadoop, HIPI, MapReduce	Effective retrieval of similar images	Scalability vs. performance; Real-time processing capabilities	Medical diagnosis, crime investigation
[11]	Advanced CBIR Framework	Hadoop MapReduce, BoVW, Binary Search-Tree	Superior recall, F-measure, and precision	Complexity of implementation; Adaptability to non-image data	Large-scale image datasets
[12]	CBIR System for Internet Images	Hadoop MapReduce, Parallel Processing	Scalable solution for large datasets	Handling varied image qualities; User interface and accessibility	Image management in big data
[13]	Medical Image Retrieval System	Hadoop, CBIR Algorithm	Reduced retrieval time for large databases	Processing speed for urgent scenarios; Compatibility with various image formats	Healthcare, medical imaging
[14]	CBIR in Medical Image Databases	Hadoop, BEMD-GGD, BEMD-HHT	Efficient handling of large-scale image data	Algorithm specificity; Resource-intensive for smaller datasets	Medical image analysis

3.1 Mapreduce

MapReduce as a programming model is very popular for processing large data sets in the distributed computing environment. It is a fundamental component of the Hadoop framework providing a powerful and scalable way of data processing [18, 19]. The MapReduce paradigm consists of two main phases as shown in Fig. 1:

3.1.1 Map phase

- The input data is divided into smaller chunks, and each chunk is processed independently by a mapper function.
- The mapper function takes a set of input key-value pairs and produces intermediate key-value pairs as output.
- The map phase operates in a parallel and distributed manner across multiple nodes in the Hadoop cluster.
- The intermediate key-value pairs are usually stored in a temporary storage area to be processed in the next phase.

3.1.2 Reduce phase

- The reduce phase takes the intermediate key-value pairs produced by the map phase as input.
- It groups together all the intermediate values associated with the same intermediate key.

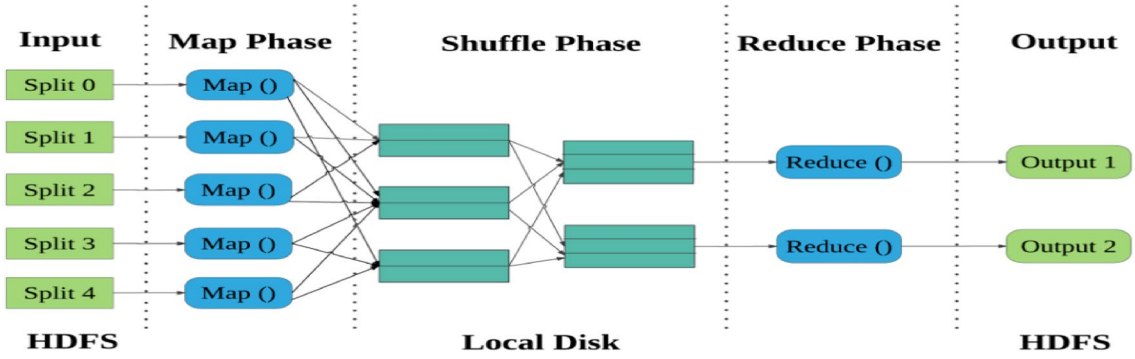

Fig. 1 The Structure of MapReduce Model

- For each unique intermediate key, a reducer function is applied to the corresponding list of values.
- The reducer function aggregates or combines the values to produce one or a few output values.
- The output values of the reducer function are the final results of the MapReduce job.
- Similar to the map phase, the reduce phase is performed in parallel and distributed across the cluster.

3.2 Hadoop distributed file system (HDFS)

HDFS is an important component of the Hadoop environment that provides highly scalable and fault-tolerant storage system for big data. It is designed to handle large datasets and to offer uniform data storage and retrieval over a set of machines [20, 21]. An overview of HDFS and its key components is provided below:

3.3 Master/slave architecture

- HDFS is a master/slave design where NameNode is the master and DataNodes are slaves.
- The NameNode controls the namespace of the file system and the meta-data of the files and directories.
- Every DataNode holds the real data blocks of the files.

3.4 Data replication and fault tolerance

- HDFS guarantees data reliability and fault tolerance by replicating the data.
- Data blocks in HDFS are replicated in multiple DataNodes to cater for failure and to achieve high availability.
- Per default, each data block is replicated to three different data nodes, which are placed on different racks to increase the fault tolerance.

3.5 NameNode and metadata management

- The metadata information about a file system, which includes file and directory names, permissions, and block locations, is stored in the NameNode.
- It maintains a record of the DataNodes that store the data blocks of each file.
- Metadata that is handled by the NameNode is stored in the memory for fast access and is also flushed to disk periodically for persistence.

3.6 Secondary NameNode

- Secondary NameNode acts as a checkpoint to NameNode.
- It sometimes gets metadata from the NameNode and combines it with the existing checkpoint data.
- The Secondary NameNode does not act as a stand-by or backup of the NameNode but helps in minimizing the recovery period in case the NameNode fails.

3.7 Hadoop cluster

- An HDFS cluster is a set of several machines, among which one is the Master node, and the other are the Worker nodes.
- The Master node consists of NameNode and ResourceManager.
- The NameNode is a file system metadata manager, and the ResourceManager is the cluster resource allocation and job scheduling component.
- Each Worker node contains a DataNode that holds the actual data blocks and a NodeManager that deals with resources and containers on that node.

When a user reads data or writes to the HDFS, he interacts with the NameNode. The NameNode returns the required information regarding the data block locations and orchestrate read/write operations. The allocation and fetching of data blocks are done by the DataNodes in accordance with the directions issued by the NameNode. HDFS is a big data application storage system that is scalable and reliable. HDFS provides data availability and durability by distributing data blocks across many machines and replicating them for fault tolerance. The NameNode is the center for metadata management, while DataNodes store and retrieve data. The Secondary NameNode acts as a checkpoint for the NameNode, which results in a shorter recovery time in case of failures [22]. The multi-node cluster is displayed in Fig. 2.

4 Methodolgy

The proposed Hadoop framework for CBIR consists of an offline training phase and an online testing phase as illustrated in Fig. 3. The offline phase involves building a feature database from images stored in the HDFS. This is done using two MapReduce jobs.

Table 2 details the steps involved in the offline phase. The first job, SequenceFiles Creation, optimizes storage by combining multiple small image files into larger sequence files [23]. In this process, the map function reads image files, creating key-value pairs with image identifiers and data, which are then shuffled, sorted, and reduced into larger sequence files for efficient storage in HDFS. The second job, Feature Extraction, processes these sequence files to extract visual features using a specified method, producing a feature database containing image identifiers and their corresponding

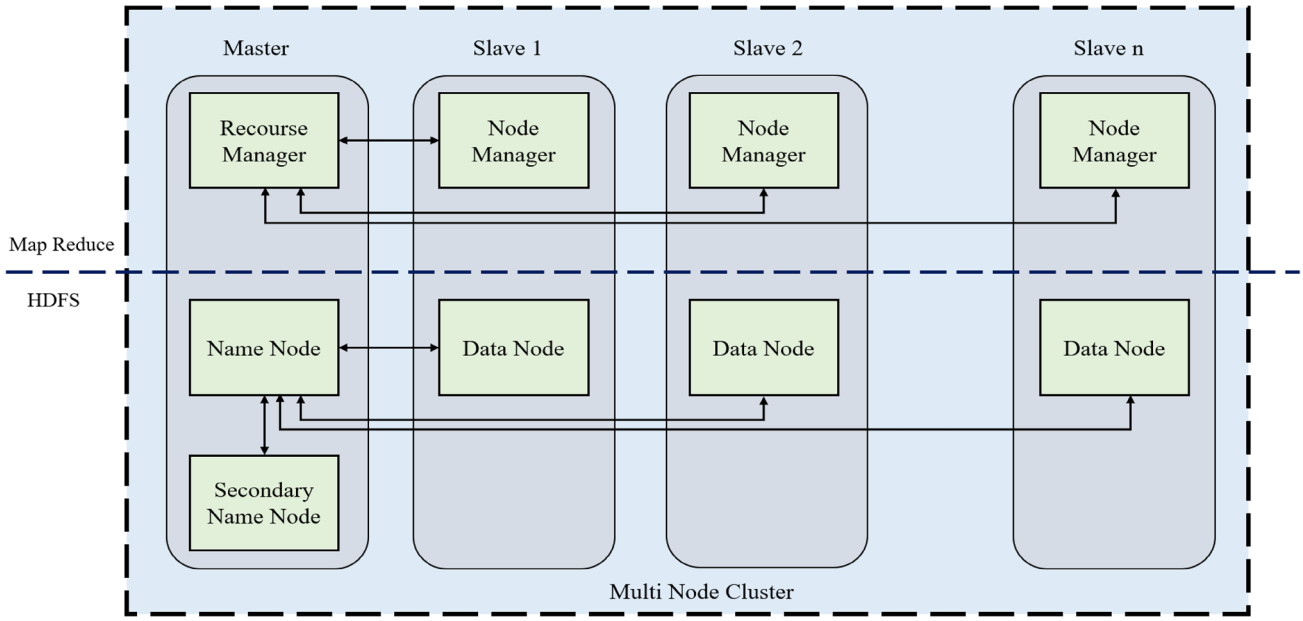

Fig. 2 The multi node cluster

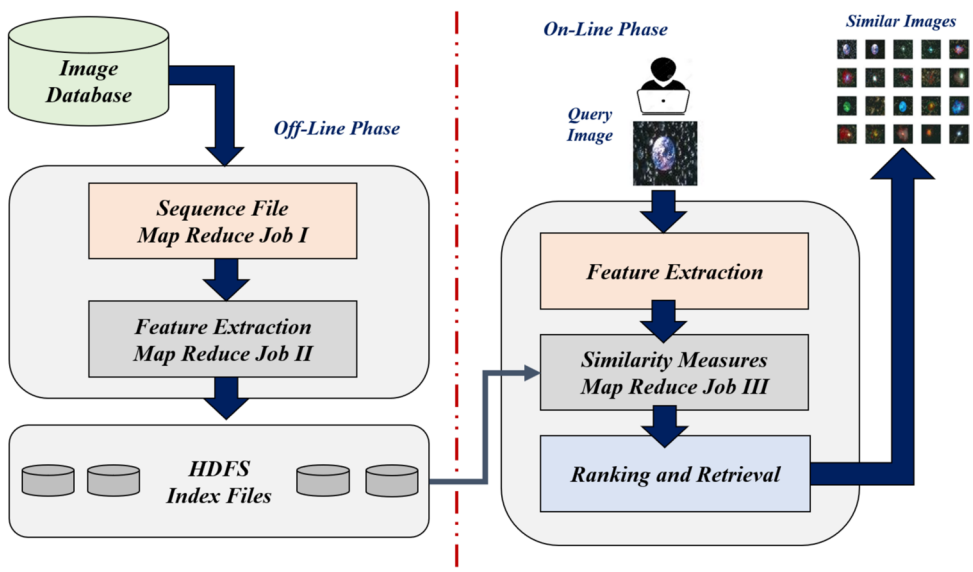

Fig. 3 Flowchart of proposed Hadoop based CBIR architecture

feature vectors. This database is stored in HDFS to ensure distributed storage, scalability, and fault tolerance.

In the online phase, described in Table 3, images are retrieved based on a query provided by the user. This phase involves a single MapReduce job that calculates the similarity between the query image and those in the feature database. The query image is processed on a server node to extract its feature vector using the same method employed during the offline phase. The map function in this job computes the similarity distances between the query image and each image in the database using a specified metric. The resulting key-value pairs, consisting of image identifiers and similarity distances, are aggregated to identify the top similar images. The server node then ranks these images, displaying the top "k" most similar images to the user for evaluation.

Table 2 Steps Involved in the Offline Phase of the Proposed Hadoop-based CBIR Method

Step		Description
Input		Image database stored in HDFS
Output		Feature database stored in HDFS
Step 1: Sequence File Creation (MapReduce Job I)	1.1 Map Function	Read small image files from HDFS and create key-value pairs: Key = Image identifier (e.g., file name), Value = Image data
		Pass the key-value pairs to the next stage
	1.2 Shuffle and Sort Phase	Redistribute key-value pairs to minimize the number of files and increase their size
	1.3 Reduce Function	Combine multiple key-value pairs into a single sequence file of a specified size (e.g., 64 MB)
		Store the sequence files in HDFS for further processing
Step 2: Successive Feature Extraction (MapReduce Job II)	2.1 Map Function	Input: Sequence files generated in Step 1. For each (image_name, image_data) pair, extract features using the specified method (FDLNP)
		Create key-value pairs: Key = Image identifier, Value = Extracted feature vector
	2.2 Shuffle and Sort Phase	Redistribute key-value pairs for further processing
	2.3 Reduce Function	Aggregate the (image_name, feature_vector) pairs into the feature database. Output the final feature database containing all image identifiers and vectors
		Store the feature database in HDFS to ensure distributed storage, fault tolerance, and scalability
End of Offline Phase		The generated feature database is stored in HDFS for use in the online (retrieval) phase

Table 3 Steps Involved in the Online Phase of the Proposed Hadoop-based CBIR Method

Step		Description
Input:		Query image and the feature database stored in HDFS
Output:		Ranked list of images similar to the query image
On Server Node	1. Extract Features from the Query Image	Receive the query image from the user
		Apply the same feature extraction method (FDLNP) used in the offline phase to obtain the query image's feature vector
		Store the extracted feature vector in memory for subsequent processing
Step 2: Similarity Distance Calculation (MapReduce Job III)	2.1 Map Function	Retrieve the feature database from HDFS. For each (image_name, feature_vector) in the feature database, compute similarity distance with the query image
		Create key-value pairs: Key = Image identifier, Value = Calculated similarity distance
	2.2 Shuffle and Sort Phase	Redistribute the similarity distance pairs for aggregation
	2.3 Reduce Function	Combine the similarity distances to generate a final list of (image_name, similarity_distance) pairs
		Store the computed similarity distances for ranking
On Server Node	3. Ranking and Retrieval	Sort the list of (image_name, similarity_distance) pairs in ascending order of similarity distance [24], using Eq. (1)
		$D_L(Q, DB) = \sum_{i=1}^{L} \left\lvert \frac{f_{DB_{ji}} - f_{Q_i}}{1 + f_{DB_{ji}} + f_{Q_i}} \right\rvert$ (1)
		Where $f_{DB_{ji}}$ is the ith feature of the jth image in the database, and f_{Q_i} is the feature vector of the query image
		Select the top "k" images with the smallest similarity distances
		Present the top "k" similar images to the user in the form of a visual plot
		The most similar images to the query image are displayed to the end user

Table 4 Steps of the FDLNP Feature Extraction Method

Step	Description	Equations
Input	Image (color or texture) of varying size	
Output	Comprehensive feature vector (FV) of size 1556 for each image	
Step 1: Image Resizing	Resize the image to uniform dimensions: 256×256 for color images and 128×128 for texture images	Let $f(x,y) = \{f_R(x,y), f_G(x,y), f_B(x,y)\}$ be an image block, where $x = 1, 2...M/16$ and $y = 1, 2...N/16$
Step 2: Min and Max Quantizers	Partition the image into 16×16 non-overlapping blocks For each block, compute the minimum and maximum quantizer values for RGB color space	Min and Max quantizers calculate using Eq. (2) and (3) respectively $q_{min}(x,y) = \left\{min_{\forall x,y} f_R(x,y) + min_{\forall x,y} f_G(x,y) + min_{\forall x,y} f_B(x,y)\right\}/3$ (2) $q_{max}(x,y) = \left\{max_{\forall x,y} f_R(x,y) + max_{\forall x,y} f_G(x,y) + max_{\forall x,y} f_B(x,y)\right\}/3$(3)
Step 3: Right Neighbor Euclidean Coding (RNEC)	For each block, compute the RNEC matrix by calculating the Euclidean distance between the center pixel (cp) and its right neighbors (RN_1, RN_2, RN_3) for both min and max quantizers	RNEC calculate using Eq. (4) $RNEC(cp) = \sqrt{(RN_1 - cp)^2 + \left(RN_2 - cp\right)^2 + \left(RN_3 - cp\right)^2}$(4)
Step 4: Local Binary Pattern (LBP)	Use the $RNEC$ matrix to compute the LBP matrix. For each pixel in the $RNEC$ matrix, calculate the LBP [25] value using the center pixel and its eight neighbors as shown in Eq. (5)	$LBP = \sum_{p=1}^{8} 2^{(p-1)} \times f_1(g_p - g_c)$ $f_1(x) = \begin{cases} 1 x \geq 0 \\ 0 otherwise \end{cases}$(5)
Step 5: Directions Matrix	Calculate the direction for each pixel in the min and max quantizer matrices based on the right neighbor pattern. The direction of the center pixel is defined as the direction from the smallest element in the pattern to the largest element in the pattern. If there are multiple minimum (maximum) elements in the pattern, we consider the first occurrence as the smallest (largest) value. Store these directions in the dir_Matrix	Direction determination based on neighbor values, as shown in Fig. 4
Step 6: Full Directions Matrices	Derive 64 matrices for each pair of direction values (dir_value, Nbr_value) where: dir_value is the direction value of the center pixel of 3×3, and its value ranges from one to eight Nbr_value is the direction value of the 3×3 neighbors around dir_value, and their values range from 1 to 8 Replace pixels in the matrices based on neighbor values and compute the decimal value for each matrix as shown in Eq. (6)	$decimal_Value = \sum_{p=1}^{8} 2^{p-1} f_{dir_value}(i)$ $f_{dir_value}(i) = \begin{cases} 1; i = Nbr_value \\ 0 Otherwise \end{cases}$(6)
Step 7: Gray-Level Co-occurrence Matrix (GLCM)	Compute GLCM [26] for each of the 65 matrices using a distance of $d = 1$ and direction $\theta = 0^{\circ}$. Extract 10 features from each GLCM matrix to capture texture patterns	Contrast, cluster prominence, entropy, sum of square variance, sum average, sum entropy, difference variance, difference entropy, information measure of correlation, inverse difference moment normalized
Step 8: Local Neighbor Pattern (LNP) Histogram	Construct the LNP histogram [27] from the LBP matrix to capture and quantify local texture variations. This forms the second feature vector (FV_2) of size 256	
Step 9: Fusion of Features	Combine the GLCM-based feature vector (FV_1) and LNP-based feature vector (FV_2) using linear combination as shown in Eq. (7). The final feature vector (FV) is of size 1556	Fusion: $FV = \alpha \times FV_1 + \beta \times FV_2$ (7) Parameters: $\alpha = 1, \beta = 0.1$ Concatenate the feature vectors:$1300 + 256 = 1556$

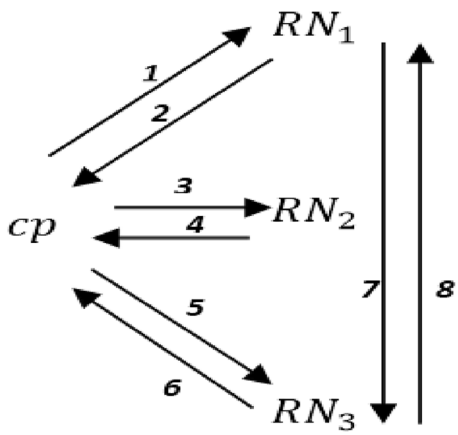

Fig. 4 Eight Directions in Neighbor Pattern

4.1 FDLNP feature extraction using hadoop mapreduce

The FDLNP is the proposed method for feature extraction within the CBIR system. This method involves multiple steps that comprehensively capture images' color, texture, and directional features to facilitate accurate retrieval. Table 4 outlines each step of the FDLNP feature extraction process in detail. To efficiently process these steps on large datasets, the FDLNP method is integrated with the Hadoop MapReduce model. Table 5 provides an overview of how FDLNP is implemented using MapReduce, detailing how each step of the feature extraction is mapped and reduced within the Hadoop environment. Figure 5 illustrates the flow of the FDLNP method within the MapReduce framework, showing the sequence of operations for feature extraction across successive MapReduce jobs.

5 Experimental setup and performance measures

The experiments were conducted using the MATLAB MapReduce framework and the HDFS, leveraging the combined power of Hadoop MapReduce and MATLAB's Parallel Computing Toolbox. In MATLAB, the MapReduce implementation integrates seamlessly with HDFS, facilitating efficient data retrieval and processing from the distributed file system. Table 6 provides the cluster configuration details, highlighting the Hadoop cluster setup used for the CBIR implementatio6n.

The metrics commonly used to measure the performance of CBIR systems are precision, recall, and mean average precision (mAP). These measures give a numeric evaluation of the system's performance in the retrieval of relevant images.

Precision (P), Recall (R), and Mean Average Precision (mAP) [28–30] are mathematically calculated as shown in Eqs. (8) - (10) respectively, knowing that a higher value of those measures reflects a better efficient retrieving system.

$$P\left(I_q,n\right)=\frac{1}{n}\sum_{i=1}^{|DB|}\varphi\left(\gamma\left(I_i\right),\gamma\left(I_q\right)\right);Rank\left(I_i,I_q\right)\le n \tag{8}$$

$$\varphi(x,y)=\begin{cases}1;x=y\\0;else\end{cases}$$

where n is the number of retrieved images, $|DB|$ is the size of the database, γ(x) is the class of the image x, and $Rank\left(I_i,I_q\right)$ returns the rank of the image I_i for the query image I_q among all images in the database.

$$R\left(I_q,n\right)=\frac{1}{N_C}\sum_{i=1}^{|DB|}\varphi\left(\gamma\left(I_i\right),\gamma\left(I_q\right)\right);Rank\left(I_i,I_q\right)\le n \tag{9}$$

where N_c indicated the max number of images in the database's class.

$$mAP=\frac{1}{|DB|}\sum_{i=1}^{|DB|}P(I_i,n);n=K_i \tag{10}$$

6 Experimental results and discussion

6.1 First experiment: evaluate FDLNP based CBIR using one local node

The first experiment aimed to assess the effectiveness of the FDLNP based CBIR system using a local node for computation on Corel 10K database. Where the Corel 10k [31] has 10,000 color images organized into 100 categories and each category contains 100 images. This large and varied database, which covers numerous topics, such as animals, landscapes, people, and buildings, is very useful for testing the robustness and scalability of CBIR systems. Most of the images are 384 × 256 or 256 × 384 pixels to make the size for analysis constant.

For evaluation Precision (K = 10), Recall (K = 100), and mAP (considering K = 10, 20, 30…100 and taking the average) are used, and FDLNP method was compared with the following state-of-the-art methods:

- Local Derived Pattern (LDP) [32], Local Tetra Pattern (LTrP) [30], Local Octa Pattern (LOtP) [33], Local Hexadecimal Pattern (LHDP) [33] and LNP [27].
- The Histogram of Oriented Gradient (HOG) method [34] calculates the gradient direction for each block in the

Table 5 Implementation of FDLNP in the MapReduce Model for Distributed Processing

Job	Phase	Steps	Output
Job 1: Initial Quantization and Grayscale Conversion	Map Phase	1. Read image data from sequence files in HDFS 2. Divide each image into blocks 3. Calculate minimum and maximum quantizer values for each block 4. Convert the image to grayscale	Key: Image identifier. Value: Processed image data (quantizer values, grayscale image)
	Shuffle & Sort	Redistribute key-value pairs for load balancing	
	Reduce Phase	Pass key-value pairs without modification for use in the next job	
Job 2: Local Pattern Analysis and Directional Encoding	Map Phase	1. Input: Processed image data from Job 1 2. Calculate LNP matrix and generate LNP histogram (size 256) 3. Compute Right Neighbor Euclidean Coding (RNEC) matrix 4. Derive the LBP matrix from RNEC 5. Generate direction and full direction matrices (64 matrices per quantizer)	Key: Image identifier. Value: Processed data (LNP histogram, LBP matrix, full direction matrices)
	Shuffle & Sort	Redistribute processed data for further computations	
	Reduce Phase	Pass key-value pairs without modification for use in the next job	
Job 3: GLCM Feature Extraction	Map Phase	1. Input: Processed data from Job 2 (LNP histogram, LBP matrix, direction matrices) 2. Calculate 130 GLCMs using the LBP and direction matrices 3. Extract 10 features from each GLCM to form a feature vector of length 1300	Key: Image identifier. Value: GLCM-based feature vector (length 1300)
	Shuffle & Sort	Redistribute feature vectors for further processing	
	Reduce Phase	Pass key-value pairs without modification for use in the next job	
Job 4: Fusion of Features	Map Phase	1. Input: GLCM-based feature vectors from Job 3 and LNP histogram from Job 2 2. Weight the LNP histogram (size 256) by a factor of 0.1 3. Concatenate the weighted LNP histogram with the GLCM feature vector to form the final feature vector (length 1556)	Key: Image identifier. Value: Final feature vector (length 1556)
	Shuffle & Sort	Organize the final feature vectors for storage in HDFS	
	Reduce Phase	Aggregate and store the < Image Number, Feature Vector > pairs in HDFS	Feature Database in HDFS

image, captures shape or edge information, and represents this using a histogram of these directions.

– Bag of Visual Words (BoVW) method [35], detects points of interest in the image and extracts features from these points, and creates a "visual dictionary", which is a dictionary of visual words, and the images are displayed as a histogram of visual word occurrences.
– Error Diffusion Block Coding (EDBTC) [36] and Multi-Integration Feature Model (MIFM) [37] methods: These

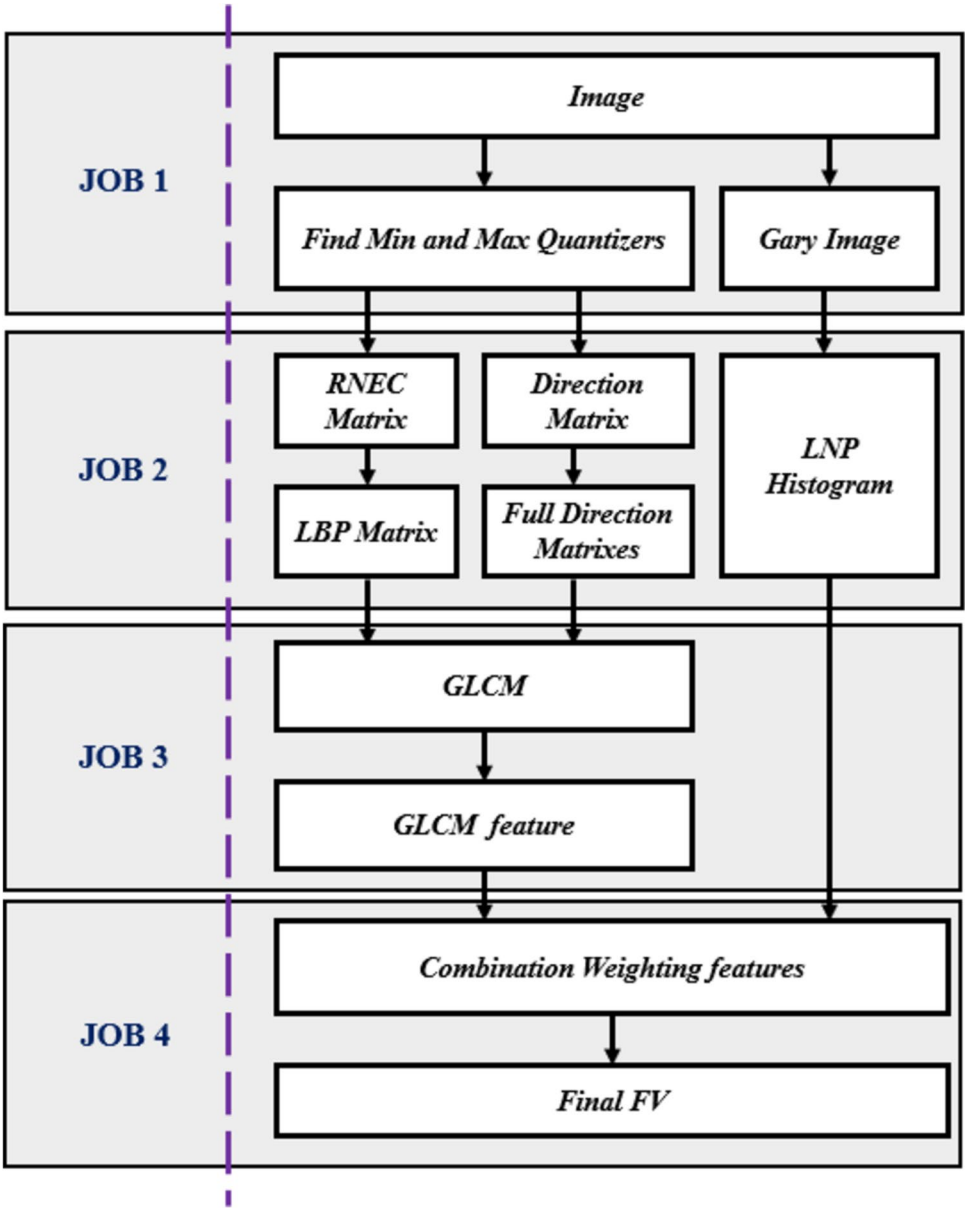

Fig. 5 Flowchart of FDLNP Feature Extraction Method in the MapReduce Model

Table 6 The setup of the Hadoop cluster used for the CBIR implementation

Node Type	Ram Size	Processor	Processor Speed	Operating System	Hadoop Version
Master	8 GB	Intel Core I7	3.6 GB	Ubuntu (16.04)-64	2.7.2
Slave	8 GB	Intel Core I7	3.6 GB	Ubuntu (16.04)-64	2.7.2
Slave	8 GB	Intel Core I7	3.6 GB	Ubuntu (16.04)-64	2.7.2

Table 7 Comparisons between FDLNP method with other methods regarding Precision, Recall, and mAP for Corel 10K database

Corel 10 k	Methods									
Performance Measure	LDP [32]	LTrP [30	LOtP [33]	LHdP [33]	LNP [27]	HOG [34]	BOVW [35]	EDBTC [36]	MIFM [37]	FDLNP
Precision	31.89	33.41	36.71	37.21	36.39	39.12	43.04	68.24	59.6	74.11
Recall	11.86	12.17	13.22	13.98	13.56	16.2	18.25	45.42	40.24	50.31
mAP	17.11	18.25	19.87	21.21	20.31	22.46	26.16	56.98	49.26	66.27

Table 8 Execution Time Comparison for the offline phase between the local node and Hadoop Cluster in minutes

Execution Time of Off-Line Phase							
Number of images in the database							
Time (minutes)	500	1000	2000	4000	6000	8000	10,000
Local Node	4.5	8	14.5	29	39	51	63
Hadoop Cluster Node	10	15	20	25	30	35	41

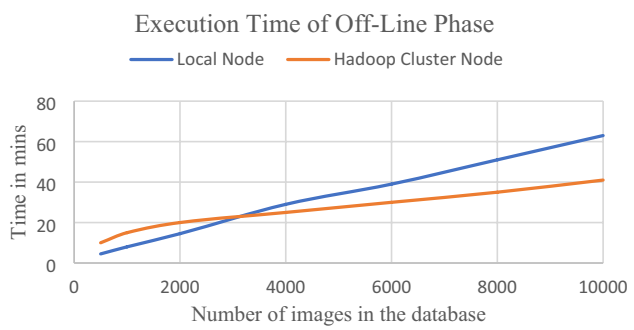

Fig. 6 Execution Time Comparison for the offline phase between the local node and Hadoop Cluster in minutes

methods extract both texture and color features for image extraction.

The results of the FDLNP method compared to other methods are depicted in Table 7.

As can be seen in Table 7, the proposed FDLNP method outperformed the comparative methods in terms of precision, recall, and mAP. This superiority is due to FDLNP's unique approach that not only extracts local pattern features, but also captures changes in color, texture, and orientation. Taking these features into account, FDLNP provides richer image representation, thus improving image retrieval performance in various databases.

6.2 Second experiment: comparative analysis of execution time for FDLNP-based CBIR on local node and Hadoop cluster

The main objective is to measure the execution time of the offline and online phases of our proposed system. To evaluate the performance, we utilized a partial Corel 10k database. However, to manage the experimental setup, we selected a subset of the database by taking only five images from each class. Starting with a small number of images, we executed the proposed system both on the Hadoop cluster and a local node. The local node refers to a single machine where the system is run without the distributed processing capabilities of the cluster. We recorded the time it took to complete the offline and online phases for each case.

In the offline phase, the execution process differs between the local node and the Hadoop cluster. On the local node, the process involves reading the image database directly from the hard drive. The local node accesses the image files, applies the proposed feature extraction method, and converts the images into a feature database. In contrast, in the Hadoop cluster, the offline phase involves not only the feature extraction step but also the steps related to data loading into HDFS, the creation of SequenceFiles, and the parallel processing of image data using MapReduce jobs.

Table 8 and Fig. 6 illustrate the comparisons of executing the proposed method on the local node and the Hadoop cluster in the offline phase. The graph displays the execution time of the proposed method for different numbers of images, ranging from a 500 up to 10,000 images.

Based on the Fig. 6, it is observed that the execution time of the proposed method on the local node is initially lower than that of the Hadoop cluster for smaller numbers of images. This can be attributed to the fact that when processing a small number of images, the Hadoop cluster incurs additional overhead in the preprocessing step for generating the Sequence File. This overhead includes tasks such as data distribution, task scheduling, and coordination among the nodes in the cluster.

On the other hand, as the number of images increases, the Hadoop cluster starts to demonstrate its advantage over the local node in terms of processing efficiency. The Hadoop

Table 9 Execution Time Comparison for the online phase between the local node and Hadoop Cluster in seconds

Execution time of on-line phase							
Online phase	Number of images in the database						
Time (sec)	500	1000	2000	4000	6000	8000	10,000
Local Node	1.25	2	3.4	4.5	5.9	6.7	7.5
Hadoop Cluster Node	2.5	3	3.6	3.9	4.1	4.4	4.8

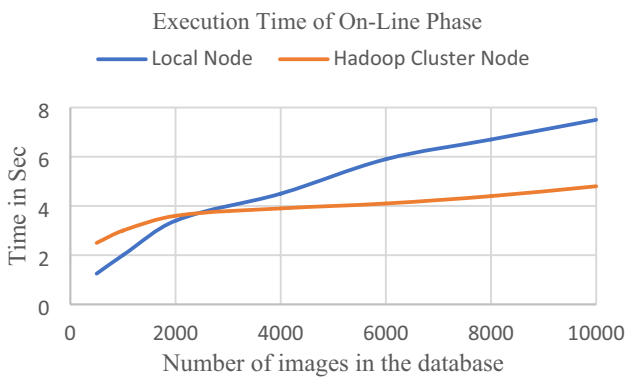

Fig. 7 Execution Time Comparison for the online phase between the local node and Hadoop Cluster in seconds

cluster spends less time in processing a larger number of images compared to the local node. This is due to the ability of the cluster to distribute the computational workload among its nodes, enabling concurrent processing and efficient resource utilization.

In the online phase, the primary objective of the cluster is to calculate the similarity distance between the query image and the images in the database. The server node, on the other hand, is responsible for extracting the features from the query image. The goal is to minimize the processing time in the online phase to provide fast retrieval of similar images to the user.

Table 9 and Fig. 7 depict a time comparison between executing the proposed method on a local node and a Hadoop cluster during the similarity measurement stage (knowing that the time is measured here by second). Similar to the offline phase, for a smaller number of images, the local node tends to perform better than the Hadoop cluster. This is because the local node only needs to extract features from the query image, which can be done efficiently on a single machine. However, as the number of images increases, the Hadoop cluster demonstrates its advantage.

In summary and based on our experiment, we recommend utilizing the Hadoop cluster for image retrieval tasks involving large datasets, where fast processing and retrieval of similar images are crucial. The local node can still be an efficient choice for smaller datasets or scenarios where scalability is not a primary concern.

Finally, Table 6 presents the execution times for both offline and online phases of our proposed model, compared with the work by Raju et al. [33] utilizing different local pattern methods (LBP, LTrP, LOtP, LHdP) on the Corel 10K dataset. While Raju et al. employed a Hadoop cluster consisting of 116 nodes (one master and 115 slaves), our model used a much smaller cluster with one master node and only two slave nodes. Despite the more modest setup,

Table 10 Execution times for different methods in offline and online phases using Hadoop clusters

Methods	Offline phase	Online phase	
	job 1	job 2	Job 3
Raju et al. [33] (LBP)	40 min	1 min 49 s	1 min 15 s
Raju et al. [33] (LTrP)	40 min	2 min 46 s	1 min 14 s
Raju et al. [33] (LOtP)	40 min	3 min	1 min 19 s
Raju et al. [33] (LHdP)	40 min	6 min 16 s	1 min 26 s
PROPOSED MODEL	41 min	4.8 s	

our proposed model significantly outperforms Raju et al.'s approach in the online phase.

It is noteworthy that while our offline phase took approximately 41 min, a duration nearest to that of Raju et al.'s methods, our process encompassed both the creation of sequence files and feature extraction for all images, storing these features in HDFS for efficient retrieval. In contrast, Raju et al.'s offline phase was limited to creating sequence files without feature extraction, which partly accounts for the comparable offline execution times. This critical difference contributes to the marked improvement in our model's online phase performance, as our approach benefits from pre-extracted features for rapid similarity measurement via a MapReduce job. This makes our method more suitable for real-time applications, as opposed to Raju et al.'s approach, which requires feature extraction from both the query image and the entire database for each retrieval task, resulting in computationally intensive and less responsive operations for real-time requirements (Table 10).

7 Conclusion

This study presents a robust solution to the challenges in managing and retrieving images from large-scale databases by integrating CBIR with the Hadoop distributed computing framework. By employing the FDLNP method for feature extraction, the experiments have demonstrated the effectiveness of this integration. In the first experiment, the FDLNP method was tested on the Corel 10K image database, achieving a precision rate of 74.11% and a mean average precision (mAP) of 66.27%. These results significantly outperform existing methods, highlighting the advantages of advanced feature extraction techniques in enhancing retrieval accuracy and efficiency.

The second experiment evaluated execution times for varying dataset sizes, ranging from 500 to 10,000 images. While smaller datasets initially favored processing on a local node due to the preprocessing overhead associated with the Hadoop cluster, the scalability and efficiency of the Hadoop

cluster became evident as dataset sizes increased. When processing a 10,000-image dataset, the Hadoop cluster outperformed the local node, showcasing its ability to handle large volumes of data efficiently. However, for smaller datasets, the local node proved more efficient. Thus, the choice between a Hadoop cluster and a local node for CBIR should depend on factors such as dataset size, available computational resources, and response time requirements.

Future advancements in CBIR systems will focus on integrating deep learning with traditional feature extraction methods to create richer image representations and enhance retrieval accuracy. This fusion will allow CBIR to capture both low-level features, like color and texture, and high-level semantic content, leading to more precise results. Incorporating user feedback will introduce a dynamic, user-centered refinement process, adapting retrieval results based on individual preferences and interactions.

Furthermore, a shift from Hadoop to Apache Spark is anticipated to greatly boost processing efficiency. Spark's in-memory computation and streamlined processing capabilities will facilitate real-time image analysis and the seamless integration of complex deep learning models. Collectively, these enhancements aim to push the boundaries of current CBIR technologies, making image retrieval more accurate, efficient, and personalized for diverse applications in sectors like healthcare, digital libraries, and surveillance.

Author contributions The authors, Maher Alrahhal and Supreethi K P, contributed equally to the work.

Funding Not applicable.

Data availability Mentioned in the paper.

Code availability Not applicable and could be provided.

Declarations

Conflict of interest The authors confirm no conflict of interest.

Research involving human and/or animals Not applicable.

Informed consent Not applicable.

References

1. Kakulapati V, Pentapati V (2022) A textual framework for contour retrieval using sub-multiple contour striking spread position learning. Int J Inf Technol 14:1575–1583. https://doi.org/10.1007/s41870-020-00425-9
2. Agarwal M (2023) Neighborhood ternary co-occurrence for natural and texture image retrieval. Int J Inf Technol 15:1999–2006. https://doi.org/10.1007/s41870-023-01238-2
3. Tamilkodi R, Nesakumari GR (2022) Image retrieval system based on multi feature extraction and its performance assessment. Int J Inf Technol 14:1161–1173. https://doi.org/10.1007/s41870-020-00556-z
4. Kanaparthi SK, Raju USN (2022) Content based image retrieval on big image data using local and global features. Int J Inf Technol 14:49–68. https://doi.org/10.1007/s41870-021-00806-8
5. Ahmad K, Sahu M, Shrivastava M et al (2020) An efficient image retrieval tool: query-based image management system. Int J Inf Technol 12:103–111. https://doi.org/10.1007/s41870-018-0198-9
6. Dong J, Zhang S, Li X, Zhou B (2018) An efficient content-based image retrieval algorithm based on Hadoop. In 2018 IEEE 2nd Advanced Information Management, Communicates, Electronic and Automation Control Conference (IMCEC) (pp. 816–820). IEEE
7. Liu Y, Chen Z, Li W (2018) An improved Hadoop-based content-based image retrieval method. In 2018 17th IEEE International Conference on Machine Learning and Applications (ICMLA) (pp. 220–225). IEEE
8. Huang Y-F, Wu H-Y (2020) Image retrieval based on ASIFT features in a Hadoop clustered system. IET Image Proc 14(1):138–146. https://doi.org/10.1049/iet-ipr.2019.0229
9. Dhulavvagol PM, Gadagkar A, Ateeth KJ, Hegade G, Poonia R, Totad SG (2023) Optimised image storage and retrieval on Hadoop. ITM Web Conf. https://doi.org/10.1051/itmconf/20235303001
10. Kavitha BR, Kumaresan P, Govindaraj R (2017) Content based image retrieval using Hadoop and HIPI. In Proceedings of the 10th International Conference on Recent Development in Engineering Science, Humanities and Management (REDHSM-17)
11. Yousefnezhad M, Moghaddam MV, Mohammadi-Ivatloo B (2016) A novel hybrid method for power system fault classification based on a new method of feature extraction. Comput Electr Eng 54:170–182. https://doi.org/10.1016/j.compeleceng.2016.04.015
12. Hinge S, Gaikwad M, Chincholkar S (2014) Content-based image retrieval using HADOOP map reduce. Int J Compu Sci Trends Technol 2(6):94
13. Grace RK, Manimegalai R, Kumar SS (2014) Medical image retrieval system in grid using Hadoop framework. In 2014 International Conference on Computational Science and Computational Intelligence (pp. 144–148)
14. Jai-Andaloussi S, Elabdouli A, Chaffai A, Madrane N, Sekkaki A (2013) Medical content based image retrieval by using the HADOOP framework. In 2013 ICT 2013 Conference Proceedings. https://doi.org/10.1109/ICTEL.2013.6632112
15. Zeng J, Li H, Gao J, Luo X (2019) A distributed K-means clustering algorithm based on Hadoop for big data. Concurr Comput 31(7):e4927
16. Wang C, Luo H, Wei J, Yu S, Liu Y, Wei, Q. (2019). A big data processing method based on Spark and Hadoop for meteorological disaster analysis. In 2019 IEEE International Conference on Artificial Intelligence and Computer Applications (ICAICA) (pp. 316–319). IEEE.
17. Li Y, Liu Y, Zhao H, Zhang Y (2019) Distributed collaborative filtering recommendation algorithm based on Hadoop. Concurr Comput 31(13):e5157
18. Alam S, Tripathy BK (2019) A comparative study of MapReduce-based algorithms for big data processing. Int J Data Sci Anal 7(4):363–378
19. Mubarak M et al (2019) An efficient parallel approach for big data processing using Hadoop MapReduce. Comp Mater Cont 59(3):945–957
20. Zainuddin NH, Sulong G, Ismail R (2017) A review of Hadoop distributed file system (HDFS) architecture and its

vulnerabilities. In 2017 International Conference on Computer and Drone Applications (ICCDA) (pp. 91–96). IEEE.
21. Xu X, et al. (2020). Secure Hadoop Distributed File System Based on Data Encryption. In Proceedings of the 2020 5th International Conference on Computer and Communication Systems (ICCCS) (pp. 35–39). ACM.
22. Apache Hadoop Documentation. (2021). SequenceFile. https://hadoop.apache.org/docs/current/api/org/apache/hadoop/io/SequenceFile.html
23. Venkateswarlu N, Reddy VR (2020) Performance analysis of MapReduce with sequence file compression techniques. Int J Inf Eng Electron Bus 12(2):35–44
24. M. Alrahhal and S. K. P, Full Direction Local Neighbors Pattern (FDLNP), International Journal of Advanced Computer Science and Applications (IJACSA), vol. 12, no. 1, 2021.
25. Pawar P, Belagali P (2015) Image retrieval technique using local binary pattern (LBP). Int J Sci Res 4(7):1440–1443
26. Islam M, Kundu K, Ahmed A (2014) Texture feature based image retrieval algorithms. Int J Eng Tech Res 2:169–173
27. Alrahhal M, Supreethi KP. Content-based image retrieval using local patterns and supervised machine learning techniques, 2019 amity international conference on artificial intelligence (AICAI), Dubai, United Arab Emirates, 2019, pp. 118–124.
28. Grossman DA, Frieder O (2004) Information retrieval: algorithms and heuristics. Springer, Cham
29. Kashyap S, Mahajan N (2021) Texture-based image retrieval using deep learning: a comprehensive survey. Wiley Interdiscip Rev 11(5):e1393
30. Murala S, Maheshwari R, Balasubramanian R (2012) Local tetra patterns: a new feature descriptor for content-based image retrieval. IEEE Trans Image Process 21(5):2874–2886
31. Content Based Image Retrieval/Image Database Search Engine (SIMPLIcity, WIPE, Virtual Microscope), Wang.ist.psu.edu, 2021. http://wang.ist.psu.edu/docs/related/
32. Zhang B, Gao Y, Zhao S, Liu J (2010) Local derivative pattern versus local binary pattern: face recognition with high-order local pattern descriptor. IEEE Trans Image Process 19(2):533–544
33. Raju U, Kumar SK, Haran P, Boppana R, Kumar N (2019) Content-based image retrieval using local texture features in distributed environment. Int J Wavelets Multires Inf Processing 18:1941001
34. Sarkar A, Das N, Content-Based Image Retrieval Using Histogram of Oriented Gradients and Local Binary Pattern, in 2016 International Conference on Communication and Signal Processing (ICCSP), Melmaruvathur, India, 2016, pp. 919–922.
35. Suharjito A, Santika DD (2017) Content based image retrieval using bag of visual words and multiclass support vector machine. ICIC Expr Lett 11:1479–1488
36. Jing-Ming J, Prasetyo H, Chen J (2015) Content-based image retrieval using error diffusion block truncation coding features. IEEE Trans Circuits Syst Video Technol 25:466–481
37. Chu K, Liu G (2020) Image retrieval based on a multi-integration features model. Math Problems Eng 2020:1–10

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.